DepthLane: Pipelined GPU–FPGA Split Inference for Real-Time Metric Depth Completion

Anonymous Author(s)
Anonymous Institution
Anonymous, Anonymous

Abstract

Metric depth completion converts sparse or noisy range measurements into dense metric geometry. Running it directly on an embedded SoC keeps sensor data local, avoids cloud round trips, and provides low-latency geometric information for applications such as autonomous driving and mobile robotics. However, existing metric depth completion models are predominantly evaluated on discrete GPUs, and few studies demonstrate high-frame-rate execution on resource- and power-constrained embedded SoCs. This leaves a practical gap between model-level accuracy and real-time edge deployment. We present DepthLane, a heterogeneous inference system that executes the RGB and RGB-D encoders on an NVIDIA Jetson AGX Thor and the decoder on an Intel Stratix 10 MX FPGA connected through four high-speed Ethernet lanes. Compared with executing the complete model on Thor, DepthLane achieves a 1.41× end-to-end speedup while maintaining the SOTA metric-depth accuracy among other depth completion model. It also reduces total energy per output frame by [TODO: XX%] relative to the all-on-Thor baseline. The prototype further demonstrates that a complete model need not reside on one processor: a compute-intensive module can be moved to an FPGA behind a well-defined tensor interface, providing a reusable path for accelerating compatible modules from other models.

Keywords

heterogeneous computing, FPGA, edge inference, depth completion, post-training quantization, split inference, frame pipelining

1 Introduction

Metric depth estimation is a key building block for spatial understanding, robotics, measurement, and scene reconstruction. Modern encoder–decoder networks provide strong accuracy, but their computational and memory demands make sustained real-time inference difficult on an embedded processor. A heterogeneous GPU–FPGA platform offers complementary resources: the GPU is well suited to the irregular, high-level feature extraction of an encoder, whereas the FPGA can implement the regular convolutional dataflow of a decoder.

This work studies AS-Depth as a concrete split-inference case. The model consumes RGB and sparse or noisy depth and produces a dense metric-depth map. Its transformer-based encoder executes on an NVIDIA Jetson AGX Thor, while its convolutional decoder executes on an Intel Stratix 10 MX FPGA. The two devices exchange intermediate features and depth outputs over four optical Ethernet lanes. The decoder is quantized to INT8 using PTQ so that it fits the FPGA compute and memory budget without requiring retraining.

Simply moving the decoder to the FPGA does not guarantee higher throughput. For a single frame, encoder execution, feature transfer, FPGA decoding, and result return remain on the end-to-end critical path. Across consecutive frames, however, these stages use independent engines. Once the features of frame i leave Thor, the GPU can encode frame $i+1$ while the FPGA decodes frame i. DepthLane uses double buffering to turn this inter-frame independence into a steady-state pipeline that hides much of the FPGA-side latency.

This paper makes the following contributions:

- We implement AS-Depth as a two-device pipeline: Thor executes the encoders, while the FPGA executes an INT8 PTQ decoder. Overlapping the encoder of frame $i+1$ with the decoder of frame i yields a 1.41× end-to-end speedup over the all-on-Thor baseline while keeping relative metric-depth degradation below 5%.
- We demonstrate the feasibility of module-level FPGA offload through a buffered tensor interface. Because the controller depends on the module's inputs, outputs, and completion signal rather than AS-Depth semantics, it can be reused for compatible compute-intensive modules from other models.
- We evaluate power at the complete-system boundary, including Thor, the FPGA board, and the interconnect, and reduce energy per output frame by [TODO: XX%] relative to the all-on-Thor baseline.

Scope. We do not propose a new depth network or target a domain-specific application stack. AS-Depth is the case study used to evaluate a module-level heterogeneous pipeline. Generalization requires a valid model partition, sufficient link bandwidth, and an FPGA-compatible offloaded stage; Section 7 states these limits.

2 Background and Motivation

2.1 AS-Depth Split Point

At the target input size of 518 × 686, the AS-Depth encoder produces a 2048 × 37 × 49 fused feature map before the convolutional decoder. This boundary is attractive because it does not require transferring a full set of U-Net skip tensors. Static analysis of the current decoder gives approximately 11.9 million parameters and 135 GMAC per frame (one multiply–accumulate counted as one MAC). The decoder expands the patch-grid representation through multiple convolution and upsampling stages before producing metric depth.

The split also exposes an important systems constraint. The boundary tensor is approximately 3.54 MiB in INT8. Serialization alone is about 29.7 ms at 1 Gb/s, 2.97 ms at 10 Gb/s, and 0.74 ms at an ideal aggregate 40 Gb/s. Thus, the current 1 Gb/s software result is not sufficient for a low-latency system. Before the accelerator evaluation, we will distinguish physical link rate from host packet-generation limits and report effective payload throughput per lane

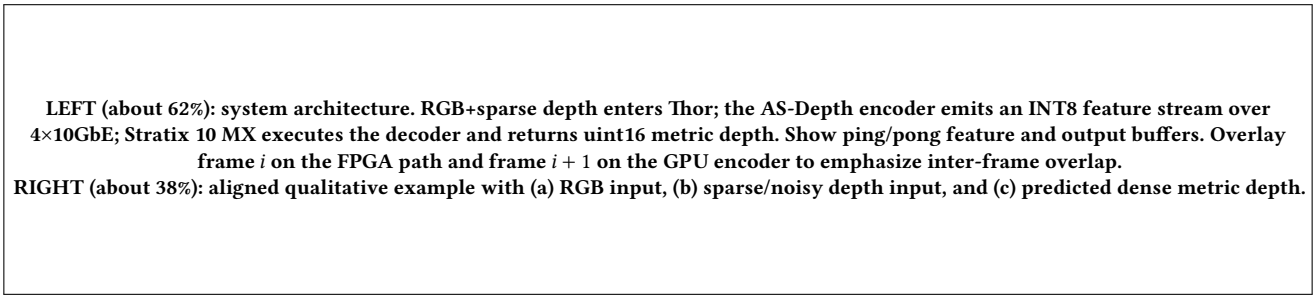


Figure 1: DepthLane overview. Consecutive frames occupy different pipeline stages: Thor encodes frame $i+1$ while frame i is transferred and decoded on the FPGA.

and in aggregate. This communication-aware framing follows the central lesson of earlier split-inference systems [6], while our focus is the pipeline opportunity created by a local GPU–FPGA system.

2.2 Inter-Frame Pipeline Opportunity

A serialized split implementation processes $E_i \rightarrow X_i \rightarrow F_i \rightarrow R_i$ before starting frame $i+1$, where E is GPU encoding, X and R are feature and result transport, and F is FPGA decoding. This schedule leaves one device idle while the other is active. With independent buffers, E_{i+1} can overlap X_i, F_i, and R_i. The steady-state frame interval is therefore governed by the slower pipeline side rather than the sum of all stage times, although the latency of an individual frame still includes every stage. Prior FPGA systems similarly use layer- or task-level pipelining to improve throughput [10, 11]; our focus is frame-level overlap across an existing GPU–FPGA model boundary.

2.3 Reusable Module Offload

The runtime treats the offloaded decoder as a module with an input tensor contract, an output tensor contract, and a completion event. It does not depend on depth-specific scheduling semantics. The same transport and double-buffer structure can therefore support another model partition if the selected module can be implemented efficiently on the FPGA and its boundary tensors fit the available link and buffer capacity. AS-Depth provides a demanding case study, not a restriction of the system architecture.

2.4 Design Goals

DepthLane has four goals: (G1) limit relative metric-depth degradation after PTQ to at most 5%; (G2) sustain at least 30 frames/s in steady state; (G3) include packing, communication, FPGA decoding, and result return in all end-to-end measurements; and (G4) quantify energy and FPGA resource cost for the complete pipeline. We treat a missed target as a measured limitation rather than reporting accelerator-kernel time alone.

3 Heterogeneous System Design

3.1 INT8 Post-Training Quantization

The decoder is quantized without retraining. We use signed INT8 weights with a separate symmetric scale for each output channel, signed INT8 activations with a static per-tensor scale, and INT32 accumulation. A calibration set of [TODO: 128 or 256] training frames determines activation ranges. We will compare simple min–max calibration with [TODO: percentile or MSE clipping] and select one method before evaluating the held-out validation set. Integer-only inference follows standard affine requantization practice [5].

Every supported operation will have a bit-exact host reference. The reference fixes rounding, saturation, accumulator width, bias scaling, and requantization before RTL validation. If a fully INT8 decoder violates the predeclared 5% relative-accuracy bound, the only fallback evaluated will be an INT16 final projection; we will not introduce QAT or a learned compression module in this paper.

3.2 Streaming FPGA Decoder

The decoder uses DSP-parallel convolution engines, INT32 partial sums, and line/tile buffers for activation reuse, following the general streaming accelerator approach exemplified by FINN [12]. Weights are loaded before the timed run and are excluded only from per-frame latency when the same exclusion is applied to GPU baselines. Consecutive layers are consumed in producer–consumer order when buffer capacity permits. The stride-2, kernel-2 transposed convolution can be implemented as an equivalent pointwise projection followed by pixel shuffle, avoiding overlap handling in the upsampling datapath.

This paper will report post-fit ALMs, DSPs, M20K bits/blocks, achieved clock, and any external-memory traffic. The existing optical design consumes about 7% of ALMs and no DSP blocks, leaving substantial arithmetic capacity, but the final compute design must close timing; "Fitter successful" is not treated as sufficient evidence.

3.3 Optical Feature Transport

Thor and the Stratix 10 MX exchange raw Ethernet frames over four 10GbE lanes. Jumbo payloads amortize packet overhead. Each packet carries a frame identifier, lane/stripe identifier, byte offset, payload length, and checksum/status field. Fixed-size striping distributes consecutive feature blocks across lanes. The FPGA writes arriving blocks into the current ping or pong buffer and exposes a frame-ready event after all expected blocks pass validation. Returned depth is encoded as a 518×686 uint16 map unless the final preprocessing pipeline selects a different padded resolution.

Execution timeline with rows for Thor encoder, optical TX/RX, FPGA decoder, and completed output. Serialized split: Enc_i, TX_i, Dec_i, and RX_i finish before Enc_{i+1}. Pipelined split: Enc_{i+1} overlaps TX_i+Dec_i+RX_i, followed by Enc_{i+2} overlapping the offload path of frame $i+1$. Mark ping/pong buffers, pipeline fill/drain, single-frame latency, and steady-state frame interval.

Figure 2: Inter-frame pipeline schedule. Thor encodes the next frame while the current frame traverses the optical link and FPGA decoder. This overlap targets throughput rather than removing stages from single-frame latency.

The transport is intentionally loss-intolerant for the evaluation: a missing or invalid packet marks the frame incomplete and removes it from useful goodput. We report offered traffic, received payload throughput, packet/frame loss, and CPU overhead. Retransmission and safety certification are future work.

3.4 End-to-End Frame Pipeline

Two feature buffers and two output buffers permit frame $i+1$ to enter the GPU while frame i is decoded. Timestamps are recorded at input release, encoder completion, final packet transmission, FPGA completion, result arrival, and application delivery. This distinguishes steady-state period from individual frame latency and prevents an overlap-only throughput gain from being reported as a single-frame latency reduction. After pipeline fill, the approximate frame interval is

$$T_{\text{frame}} \approx \max(T_E, T_{\text{offload}}), \tag{1}$$

where T_{offload} includes feature transport, FPGA decoding, and result return, including any overlap internal to that path. Sustaining 30 frames/s requires $T_{\text{frame}} \leq 33.3\,\text{ms}$.

4 Inter-Frame Pipeline Execution

4.1 Stage and Buffer Model

Each frame passes through GPU encoding E_i, feature transport X_i, FPGA decoding F_i, and result return R_i. Dependencies are enforced only within the same frame. Across frames, E_{i+1} may begin after a free feature buffer is available, without waiting for F_i or R_i to complete. Ping/pong feature and output buffers prevent the producer from overwriting data still consumed by the link or FPGA.

4.2 Pipeline Controller

The host controller maintains buffer ownership and frame identifiers. It submits E_{i+1} while the offload path processes frame i, then associates each returned depth map with its original input. Backpressure stops a new encoder launch when both feature buffers are occupied. This bounded policy avoids unbounded queues while preserving every frame during the offline evaluation. The controller is dedicated to consecutive frames of the same model and makes no multi-model admission decision.

4.3 Generality Beyond AS-Depth

The pipeline controller only requires tensor dimensions, serialization rules, buffer addresses, and completion events for the selected partition. A module from another model can replace the AS-Depth decoder when (1) its operators map efficiently to the FPGA, (2) its weights and working set fit local or HBM capacity, and (3) boundary-transfer time does not dominate its saved compute. The achievable speedup remains model- and split-dependent, so we present this as a reusable system mechanism rather than a universal acceleration claim.

4.4 Baselines

We compare four configurations. *Thor FP32* and *Thor TensorRT FP16* execute the complete model on Thor. *Serialized split* executes the Thor encoder and INT8 FPGA decoder without overlap between consecutive frames. *Pipelined split* uses identical hardware, quantization, and transport but overlaps E_{i+1} with the offload path of frame i. Comparing the last two isolates the benefit of inter-frame pipelining from model partitioning and PTQ.

5 Prototype and Methodology

Tables 1 and 2 summarize the two hardware platforms. The FPGA table reports device capacity; post-fit resources consumed by our decoder are reported separately in Table 3(b).

All input files are decoded before a timed run and replayed from memory. This removes storage variability without requiring a live sensor, while preserving real model execution, packet transfer, buffering, and synchronization. Each sample supplies the aligned RGB and sparse/noisy-depth inputs expected by AS-Depth. Accuracy runs process every sample without overlap-dependent drops; throughput runs replay the same fixed input sequence for every baseline.

Thor power is sampled from the platform's exposed GPU and system-input rails [9]. FPGA power is measured at the development-board input and cross-checked with the board tool where possible [3]. The headline energy metric includes both devices and optical transceivers. We also report active-minus-idle energy to explain static board overhead, but do not use it as a substitute for total system energy.

6 Evaluation Plan and Results

The evaluation answers three questions. **RQ1:** Does INT8 decoder PTQ keep metric-depth degradation within 5%? **RQ2:** Does the complete Thor–FPGA pipeline sustain at least 30 frames/s, and at what latency, energy, and FPGA cost? **RQ3:** How much offload-path time is hidden by inter-frame overlap relative to a serialized split?

Table 1: NVIDIA Jetson AGX Thor hardware configuration. Peak throughput and port capability are vendor specifications [7, 8]; average power is measured at the platform input while running the evaluated workload. The prototype operates the QSFP28 link at 4×10 GbE.

CPU	GPU	Peak AI throughput
14-core Arm Neoverse V3AE, 2.6 GHz	2,560-core Blackwell, 10 TPCs	1,035 INT8 TOPS; 2,070 sparse-FP4 TFLOPS
Supported precision	Average power	Optical interface
FP32/TF32/BF16/FP16/FP8/FP4; INT8; INT4 weights	[TODO: measured W] (40–130 W mode range)	1×QSFP28 (4×25 GbE); used at 4×10 GbE

Table 2: Intel Stratix 10 MX FPGA hardware configuration. Values are total device capacities rather than post-fit utilization [1]; Intel reports ALMs rather than LUTs. Average power is measured at the board input, and the two QSFP28 cages are 100GbE-capable while this prototype uses four 10 GbE lanes [2, 4].

Device	Logic resources	DSP resources and precision
1SM21BHU2F53E2VGS1 (MX 2100)	2,073,000 LEs; 702,720 ALMs; 2,810,880 registers	3,960 DSPs; 2×18×19 or 1×27×27 fixed-point; hardened FP32
Memory resources	Average power	Optical interface
6,847 M20Ks (134 Mb); 11 Mb MLAB; 94.5 Mb eSRAM; 8 GB HBM2	[TODO: measured W]	2×QSFP28 (100GbE capable); used at 4×10 GbE

Table 3: Primary accuracy, systems, and FPGA results. All numerical cells are intentionally blank until reproduced on the final software/bitstream.

(a) Depth accuracy and end-to-end performance on Hammer

Configuration	RMSE (mm)↓	MAE (mm)↓	iRMSE (1/km)↓	iMAE (1/km)↓	p50 (ms)↓	p99 (ms)↓	FPS↑	J/frame↓
Thor full model FP32	–	–	–	–	–	–	–	–
Thor full model TensorRT FP16	–	–	–	–	–	–	–	–
Serialized Thor encoder + FPGA INT8 decoder	–	–	–	–	–	–	–	–
Pipelined Thor encoder + FPGA INT8 decoder	–	–	–	–	–	–	–	–

(b) Post-fit FPGA implementation

Design	ALMs	Registers	DSPs	Memory bits / M20Ks	Fmax (MHz)	Decoder latency (ms)
Four-lane optical baseline	47,261	54,618	0	704,064 / 77	–	N/A
Optical + INT8 decoder	–	–	–	–	–	–

6.1 RQ1: Accuracy Preservation

We evaluate the complete held-out Hammer set using RMSE, MAE, iRMSE, and iMAE. The principal comparison uses the same checkpoint and preprocessing with a floating-point decoder and the INT8 PTQ decoder. For each error metric m, we report $(m_{\text{INT8}} - m_{\text{float}})/m_{\text{float}}$ and require the relative increase to remain at or below 5%. This threshold and the calibration procedure are fixed before evaluating the held-out set. One qualitative row in Fig. 1 will show a representative sample and a difficult failure case if space permits.

6.2 RQ2: Real-Time Throughput and System Cost

Table 3 reports steady-state FPS, end-to-end latency, energy, and post-fit resource use. Real-time operation is supported only if the pipelined configuration sustains at least 30 complete outputs/s, corresponding to a frame interval no greater than 33.3 ms. We separately report encoder, feature-transfer, decoder, return, and synchronization times so the source of any missed target is visible. Energy per frame is the integral of complete platform power over the run divided by the number of returned depth maps.

6.3 RQ3: Pipeline Overlap Effectiveness

Figure 3 compares serialized and pipelined split execution over the same ordered frame sequence. We report pipeline fill and drain separately from steady state, together with the fraction of offload-path time hidden behind E_{i+1}. The pipeline claim is supported only if overlap improves completed-frame throughput without changing output tensors or dropping evaluation frames. We do not claim that a 30-FPS throughput implies a single-frame latency below 33.3 ms; these are reported as distinct quantities.

Three aligned plots for Thor FP32, Thor FP16, serialized split, and pipelined split: (a) steady-state FPS with a 30-FPS target line; (b) encoder, transfer, FPGA decoder, return, and exposed-versus-hidden time; (c) average watts and joules per frame. Use identical input sequences and measured confidence intervals.

Figure 3: Planned throughput, overlap, and energy comparison. Serialized and pipelined split execution use the same model, INT8 decoder, hardware, and transport configuration, isolating the benefit of inter-frame overlap.

7 Discussion and Limitations

The prototype evaluates prerecorded inputs rather than live sensor I/O. The observed accuracy preservation is specific to the evaluated checkpoint, calibration set, and decoder; PTQ does not guarantee the same bound for another model. Insufficient effective link throughput can dominate the split even when the FPGA decoder is fast. The 30-FPS target concerns steady-state throughput after pipeline fill, not the latency of an isolated frame. Generalization to another model is also not automatic: the selected module must have supported operators, adequate parallelism, a manageable boundary tensor, and a working set that fits the FPGA memory hierarchy. Activation compression, dynamic repartitioning, and fault recovery are outside the present scope.

8 Conclusion

DepthLane is a GPU–FPGA split-inference system for real-time metric depth estimation. Thor executes the AS-Depth encoder, while a PTQ INT8 decoder runs on the FPGA. Double buffering overlaps encoder execution for the next frame with transport and FPGA decoding of the current frame, hiding offload-path time in steady state. Relative to executing the complete model on Thor, this design achieves a 1.41× end-to-end speedup while keeping relative metric-depth degradation below 5%, and reduces energy per output frame by [TODO: XX%]. Beyond AS-Depth, the buffered module interface provides a path to offload compatible stages from other models, subject to operator, memory, and communication constraints.

References

[1] Intel Corporation. 2018. Intel Stratix 10 MX FPGA Family Overview Product Table. https://cdrdv2-public.intel.com/652458/stratix-10-mx-product-table.pdf

[2] Intel Corporation. 2024. Stratix 10 Variable Precision DSP Blocks User Guide. https://www.intel.com/programmable/technical-pdfs/683832.pdf

[3] Intel Corporation. 2025. Intel Stratix 10 MX FPGA Development Kit Quick Start Guide. https://cdrdv2-public.intel.com/649826/intel_stratix_10_mx_qsg.pdf

[4] Intel Corporation. 2025. Stratix 10 MX FPGA Development Kit User Guide. https://www.intel.com/programmable/technical-pdfs/683867.pdf

[5] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2704–2713.

[6] Yoshitomo Matsubara, Sabur Baidya, Davide Callegaro, Marco Levorato, and Sameer Singh. 2019. Distilled Split Deep Neural Networks for Edge-Assisted Real-Time Systems. In *Proceedings of the 2019 Workshop on Hot Topics in Video Analytics and Intelligent Edges*. 21–26. doi:10.1145/3349614.3356022

[7] NVIDIA Corporation. 2025. Introducing NVIDIA Jetson Thor, the Ultimate Platform for Physical AI. https://developer.nvidia.com/blog/introducing-nvidia-jetson-thor-the-ultimate-platform-for-physical-ai/

[8] NVIDIA Corporation. 2026. Jetson Thor: Advanced AI for Physical Robotics. https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-thor/

[9] NVIDIA Corporation. 2026. Jetson Thor Product Family: Platform Power and Performance. https://docs.nvidia.com/jetson/archives/r39.2/DeveloperGuide/SD/PlatformPowerAndPerformance/JetsonThor.html

[10] Seyyed Ahmad Razavi, Hsin-Yu Ting, Tootiya Giyahchi, and Eli Bozorgzadeh. 2022. On Exploiting Patterns for Robust FPGA-Based Multi-Accelerator Edge Computing Systems. In *Design, Automation and Test in Europe Conference and Exhibition (DATE)*. https://past.date-conference.com/proceedings-archive/2022/pdf/0910.pdf

[11] Arne Symons, Linyan Mei, Steven Colleman, Pouya Houshmand, Sebastian Karl, and Marian Verhelst. 2022. Towards Heterogeneous Multi-Core Accelerators Exploiting Fine-Grained Scheduling of Layer-Fused Deep Neural Networks. arXiv:2212.10612 https://arxiv.org/abs/2212.10612

[12] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 65–74. doi:10.1145/3020078.3021744